



Kingdom of Saudi Arabia
Ministry of Education
King Faisal University
College of Computer Sciences & Information Technology
Department of Computer Sciences

CS 320: Computer Security

Programming Project Report

SecureVault: Secure Messaging and Local File Vault Using AES-256

Instructor: Dr. Abdullah Albuali

Group Members
Hassan Mohammed Alqahtani – 223003175
Abdulaziz Mathkar Almutairi – 223014463
Nawaf Sultan Alghamdi – 223035370

Section: 01
Second Semester 2025-2026

Project Marking Guide (Programming Project)

Project Title: SecureVault: Secure Messaging and Local File Vault Using AES-256

Assignment Component	Max. Marks	Marks Obtained
Format (Table of Contents, References, formatting and font, etc.)	10	
English language (grammar and spelling)	10	
Introduction	15	
References	10	
Contents / Algorithms / Discussions / Coding	30	
Demonstration / Questions and answers	15	
Interface Design & Usability	15	
Testing (Various Input Conditions)	15	
Presentation skills	15	
Understanding / Explanation of the code	15	
Total Marks	150	
Grade for this Project	15	

Table of Contents

1. Introduction
2. System Design (Architecture)
3. Methods / Algorithms
4. Implementation
5. Screenshots
6. Testing & Results
7. GitHub Link
8. References

AI usage declaration

AI tools were used to assist in organizing the project idea, reviewing code readability, and supporting code generation. The project group reviewed the final content, executed the code locally, tested the system, and verified the results. No real or sensitive user data was used; all testing was performed using local test data only.

1. Introduction

SecureVault is a local security application developed for the CS 320 programming project under the topic Secure Messaging / Vault. The project was designed to stay inside the guideline provided by the course instructor. The guideline requires a GUI, a GitHub-ready code repository, a live or video demo, functional testing, and safe use of local or test data only. In response to those requirements, SecureVault provides a graphical interface that encrypts messages and files, stores encrypted vault files locally, demonstrates failed attack attempts, and includes automated tests with recorded results.

The main objective of the project is to show how a practical vault-style system can protect information before storage or sharing. A message that appears readable on screen as plaintext should become unreadable ciphertext after encryption. The same idea is applied to local files in the file vault. Only a user who knows the correct password can derive the correct secret key and recover the original content. This directly supports the project idea attached in the guideline, which asks for an encryption system, key handling, and an attack demo.

The project is limited to local and test data. It does not connect to any remote service, cloud database, or real user account. This is intentional because the guideline explicitly asks students to use local or test data only. The system therefore focuses on the core security concepts required for the course: confidentiality of stored data, careful password-based key handling, user awareness during decryption attempts, and testing under multiple input conditions.

The implemented system uses AES-256 for content protection. According to the NIST AES standard, AES is a symmetric block cipher that processes 128-bit data blocks and supports key sizes of 128, 192, and 256 bits. SecureVault chooses the 256-bit option because it offers the strongest key size among the standard AES options. Although the application is small and academic in scope, it follows the same basic pattern seen in real secure storage systems: derive a key from a password, encrypt the content, store the encrypted output, and recover the original data only when the correct password is supplied.

AI usage declaration: AI assistance was used to help draft code structure, documentation, screenshots, and report wording. All generated material was manually reviewed, corrected, executed, and tested before inclusion in the final project submission.

2. System Design (Architecture)

SecureVault uses a simple layered design so that each part of the program has one clear role. The first layer is the GUI layer built with Tkinter. It is responsible for accepting user input, displaying encrypted output, letting the user choose local files, and presenting attack results in a readable way. The second layer is the cryptographic layer. It performs key derivation, AES encryption, AES decryption, and payload formatting. The third layer is the storage layer, which saves encrypted vault files to a local project folder. The fourth layer is the testing layer, which runs functional test cases and records results.

This structure was selected because it supports the project requirements without making the code unnecessarily large. The GUI remains focused on usability, while the cryptographic functions remain isolated inside a separate module. As a result, the same crypto functions can be called from both the GUI and the test suite. This makes the program easier to explain in the class presentation and easier to evaluate when discussing correctness, interface design, and testing.

Component	Role	Main Files / Objects
GUI Layer	Collects plaintext, passwords, and file selections; displays ciphertext and results.	app.py / Tkinter tabs
Crypto Layer	Derives keys and performs AES encryption/decryption.	crypto_utils.py
Vault Storage	Stores encrypted file payloads as local .svault files.	vault_data/ folder
Testing Layer	Verifies correct and incorrect behavior with repeatable cases.	tests/test_securevault.py

Table 1. SecureVault architecture summary.

3. Methods / Algorithms

The project method follows a password-to-key workflow. Users do not type an AES key directly. Instead, they enter a password in the GUI. That password is converted into a strong cryptographic key using PBKDF2-HMAC-SHA256 with a random salt and a high iteration count. This step is important because passwords chosen by people are usually easier to remember than random binary keys, but they are also weaker if used directly. Key derivation strengthens the password input before it is used by the encryption algorithm.

After key derivation, SecureVault encrypts the content using AES-256. AES is a symmetric cipher, meaning the same secret key is required for both encryption and decryption. In the message vault, plaintext is converted to bytes, padded to the correct block length, encrypted, and then stored inside a JSON payload. The payload contains the algorithm label, KDF label, iteration count, random salt, initialization vector, and ciphertext. This makes the output self-contained and easier to save, inspect, and reuse during testing.

The file vault uses the same basic method but applies it to an entire local file. The selected file is read as bytes, encrypted, and then stored in a .svault JSON file. The original filename is stored in the payload so that the recovered file can be written back with its correct name during decryption. This design keeps the implementation clear and practical for a classroom demo while still showing a real file protection workflow.

The attack demo focuses on negative behavior rather than offensive functionality. First, the program attempts decryption with a wrong password. Because the derived key is incorrect, the plaintext cannot be recovered and the program reports a failure. Second, the ciphertext can be intentionally modified in the attack tab. When the modified payload is decrypted, the operation also fails. These two cases demonstrate that protected data should not become readable when the correct secret is absent or the payload has been altered. This is a safe demonstration for a course project because it teaches defensive security behavior without using real attack tools or unauthorized data.

The test suite applies the same methods under multiple conditions. Round-trip tests confirm that encrypted data returns to its original form when the correct password is used. Negative tests confirm that wrong passwords and modified payloads fail. Additional tests verify that file encryption works, repeated encryption creates different outputs because random values are used, and empty passwords are rejected. Together, these methods satisfy the required project elements: encryption system, key handling, attack demo, and functional tests.

4. Implementation

The final implementation is organized as a small Python project. The main file, `app.py`, builds the local GUI and manages user interaction. The file `crypto_utils.py` contains the cryptographic functions such as key derivation, message encryption, file encryption, message decryption, file recovery, and payload tampering for demonstration purposes. The tests folder contains ten automated unit tests, and `run_tests.py` executes them and saves the result to `test_results.txt` for later reporting.

The interface is split into four tabs. The Message Vault tab handles text encryption and decryption. The File Vault tab handles local file selection, vault storage, and file recovery. The Attack Demo tab is dedicated to wrong-password and modified-payload experiments so that the instructor can see a controlled security scenario during the demonstration. The Project Notes tab summarizes the purpose and safe-usage rules of the application.

Implementation decisions were kept practical and easy to explain in front of the class. First, the application was built with Tkinter because it provides a native GUI without requiring external servers or cloud deployment. Second, encrypted vault files are stored inside the project folder, which keeps the solution local and compatible with the course rule about safe data handling. Third, the project includes a GitHub-ready directory structure with README, requirements file, `.gitignore`, screenshots, tests, and sample local files. This makes the project easy to upload as a repository before submission.

A short video demo was also prepared as part of the project package. The video presents the title, message encryption screen, file vault screen, attack demo screen, and functional test result screen. In addition, a `DEMO_CHECKLIST` file was included so the group can repeat the same steps during a live presentation. This supports the guideline item that asks for a live or video demo.

From a security point of view, the implementation avoids unsafe shortcuts that would weaken the educational value of the project. The password is not stored as plaintext. The AES key is derived only when needed. Each encryption operation uses fresh random values so that the same message does not produce the same output every time. Finally, the program avoids real confidential data and keeps all processing inside a local classroom environment.

5. Screenshots

SecureVault

Local AES-based secure messaging and file vault with key handling, attack demo, and functional testing.

Message VaultFile VaultAttack DemoProject Notes

Encrypt Message

Plaintext

This is a secret message:

Computer Security Section 01
password: 1234

Password

Encrypt MessageLoad SampleClear

Ciphertext / Decryption

Encrypted JSON payload

```
{  "algorithm": "AES-256-CBC",  "kdf": "PBKDF2-HMAC-SHA256",  "iterations": 200000,  "salt": "JnsKt5cq6mpt2f9RulEi3A==",  "iv": "AnTzls/t0692mrX6qQ4qRA==",  "ciphertext": "EQHic7Nt+SRqjC14cqZ7Phzu/K/1ZZ1HK0uk5WBh7WWYBYwROBAI/PKdYu/xmigiN0hwHE8hhcPT8T8uda/EH27Kx3e+47e1Geb19frL58/Y="}
```

Password for Decryption

Decrypt MessageCopy to Attack Demo

Message encrypted successfully.

Figure 1. Message Vault tab after encrypting a sample message.

SecureVault

Local AES-based secure messaging and file vault with key handling, attack demo, and functional testing.

Message Vault**File Vault**Attack DemoProject Notes

Encrypt Local File

/Users/hassan_alqahtani/Desktop/2026- Project guideline Computer Security.docx

Choose File

Password

Encrypt and Save to Vault

Vault output folder: vault_data/

Encrypted file saved: 2026- Project guideline Computer Security.svault

Decrypt File from Vault

2026- Project guideline Computer Security.svault



Recovered file: 2026- Project guideline Computer Security.docx

OK

Refresh Vault List

Password

Decrypt Selected File

Figure 2. File Vault tab showing local file encryption and vault contents.

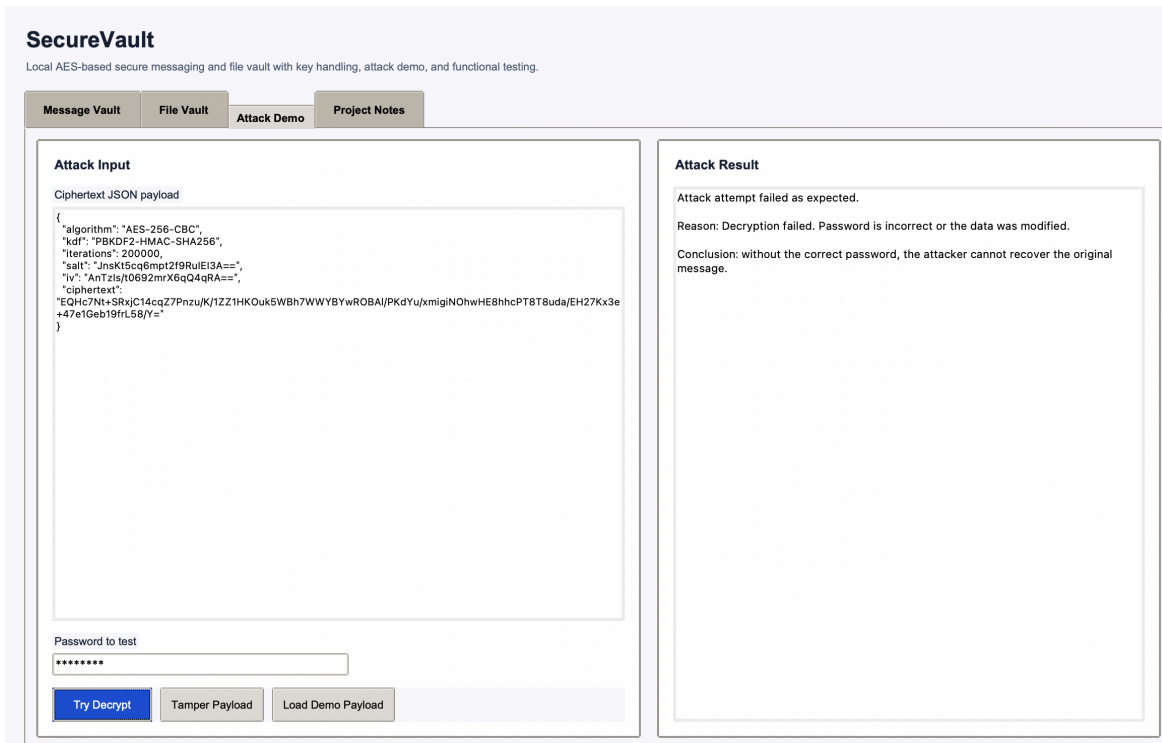


Figure 3. Attack Demo tab showing failed decryption with the wrong password.

```

hassan_alqahtani@MacBook-Pro-alkhas-bhsn SecureVault_Code_Only % python3
.....
-----
Ran 10 tests in 1.176s

OK

```

Figure 4. Functional test execution result showing all tests passed.

6. Testing & Results

Testing was performed with automated functional cases and direct GUI inspection. The goal was not only to show successful encryption, but also to confirm that incorrect input conditions are handled safely. Ten automated tests were executed and all passed successfully. The complete output is included in the project folder as test_results.txt and shown in Figure 4 above.

Test Case	Expected Result	Actual Result
Same password + same salt derive same key	Pass	Pass
Message encrypt/decrypt round-trip	Pass	Pass
Wrong password for message	Fail securely	Fail securely
Tampered payload	Fail securely	Fail securely
Byte-level encrypt/decrypt round-trip	Pass	Pass
Payload contains required fields	Pass	Pass
File encryption and recovery	Pass	Pass
Wrong password for file	Fail securely	Fail securely
Empty password rejected	Pass	Pass
Repeated encryption gives different ciphertext	Pass	Pass

Table 2. Functional test cases and observed results.

The test results show that the system behaves correctly in both positive and negative paths. Valid passwords recover the original message or file, while invalid passwords do not reveal the plaintext. Randomized outputs for repeated encryption confirm that the implementation does not produce identical ciphertext for identical inputs every time. This is an important property because repeated patterns can weaken real-world protection.

The interface also remained usable during manual checks. The separation into tabs made it easy to show one feature at a time during demonstration. Students can therefore explain the project in a structured order: message encryption first, file vault second, and attack demo third. This supports the marking criteria for interface design, usability, demonstration, and code explanation.

7. GitHub Link

The source code of the SecureVault project was uploaded to a GitHub repository to meet the project code submission requirement.

Repository URL: <https://github.com/AboM7mdy/Secure-Messaging-Vault/>

8. References

[1] National Institute of Standards and Technology, Advanced Encryption Standard (AES), FIPS PUB 197, Nov. 2001. Updated May 9, 2023.

<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>